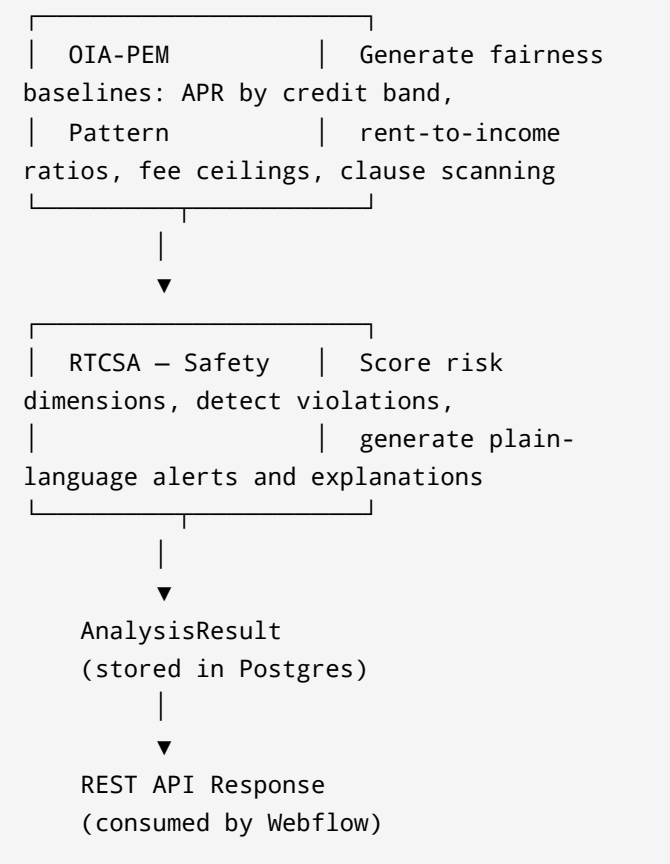
ARKHEIA Contract Protection System (CPS)

Production Backend — FastAPI + PostgreSQL + Render.com

SYSTEM OVERVIEW

ARKHEIA-CPS is a discipline-based, real-time contract safety analysis backend.
It applies the three-layer ARKHEIA pipeline to AUTO and HOUSING contracts:

```
CONTRACT INPUT (JSON)
        │
        ▼
┌──────────────────────┐
│  FIA — Identity   │  Structure all entities: consumer, counterparty,
│                   │  vehicle/property, financial terms, dates
└──────────┬────────┘
           │
           ▼
```



VERTICALS

AUTO

- Affordability (payment-to-income ratio)
- APR by credit band (EXCELLENT → DEEP_SUBPRIME)

- Fee analysis (doc fees, dealer fees, undisclosed fees)
- Add-on detection (forced, bundled, inflated GAP)
- Total cost math verification
- **Perfection Law** (lien timing, illegal sequencing)

HOUSING

- Rent-to-income ratio
- Total move-in cost (vs. monthly rent multiples)
- Fee analysis (application, admin, security deposit)
- Rent increase clauses
- Late fee and grace period evaluation
- Early termination penalties
- Automatic renewal risk
- Eviction timing analysis
- **Illegal clause detection** (habitability waivers, entry without notice, etc.)
- Mandatory recurring fee evaluation

PERFECTION LAW MODULE (AUTO)

Detects illegal and predatory lien perfection patterns:

Rule Code	Violation	Severity
PERF_MISSING	No perfection date recorded	HIGH
PERF_LATE (31–60d)	Moderate delay in perfection	MODERATE
PERF_LATE (61–90d)	High delay in perfection	HIGH
PERF_LATE (>90d)	Severe delay — may affect enforceability	HIGH
PERF_AFTER_REPO	Lien perfected after repossession	EXTREME
PERF_SAME_DAY	Perfection and repo on same date	EXTREME
REPO_NO_NOTICE	Repo before notice of default	EXTREME

Rule Code	Violation	Severity
REPO_BEFORE_CURE	Repo before right-to-cure notice	EXTREME
REPO_CURE_TOO_SOON	Cure period not fully honored	HIGH
LIEN_NOTATION_LATE	Lien notation on title > 30 days	MODERATE

DIRECTORY STRUCTURE

```
arkheia_cps/
├── app/
│   ├── main.py                    #
FastAPI entry point
│   ├── config.py                  #
Settings from env vars
│   ├── db.py                      #
SQLAlchemy engine + session
│   ├── models/
│   │   ├── common.py              #
Contract, Consumer, Counterparty,
AnalysisResult
│   │   ├── auto.py                #
AutoContractDetails (with perfection dates)
```

```
│   │   └── housing.py            # HousingContractDetails (full lease fields)
│   ├── schemas/
│   │   ├── common.py             # Shared Pydantic models
│   │   ├── auto.py               # AutoContractIn, AutoAnalysisResponse
│   │   └── housing.py            # HousingContractIn, HousingAnalysisResponse
│   ├── routers/
│   │   ├── contracts.py          # POST /contracts/auto + /contracts/housing
│   │   └── analysis.py           # GET /analysis/{id}, list, stats
│   ├── services/
│   │   ├── fia_auto.py           # FIA Auto — entity structuring
│   │   ├── fia_housing.py        # FIA Housing — entity structuring
│   │   ├── oia_pem_auto.py       # OIA-PEM Auto — pattern evaluation
│   │   ├── oia_pem_housing.py    # OIA-PEM Housing — pattern evaluation
│   │   ├── rt_csa_auto.py        # RTCSA Auto — risk scoring + alerts
│   │   ├── rt_csa_housing.py     # RTCSA Housing — risk scoring + alerts
│   │   └── perfection_law.py     # Perfection Law — lien timing analysis
│   └── utils/
│       ├── scoring.py            # Weighted score computation
│       ├── explanations.py       # Plain-language alert generator
```

```
│       ├── statutes.py            # State
statute mapping (GA, FL, TX, CA, NY)
│       └── file_extraction.py     # OCR
stubs (PDF/DOCX — future)
├── alembic/
│   └── env.py                     #
Alembic migration config
├── alembic.ini
├── requirements.txt
├── render.yaml                    #
Render.com deployment config
├── sample_payloads.json           # 3
auto + 2 housing test payloads
└── README.md
```

API ENDPOINTS

Contracts

```
POST /contracts/auto
  Body: AutoContractIn (JSON)
  Returns: AutoAnalysisResponse

POST /contracts/housing
  Body: HousingContractIn (JSON)
  Returns: HousingAnalysisResponse
```

Analysis

```
GET  /analysis/{contract_id}
  Returns: Full stored AnalysisResult

GET  /analysis?
vertical=AUTO&risk_level=RED&limit=50&offse
t=0
  Returns: Paginated list of
AnalysisResults

GET  /analysis/summary/stats
  Returns: Aggregate counts by risk level
and vertical
```

System

```
GET  /health   → {"status": "ok", ...}
GET  /         → Endpoint directory
GET  /docs     → Swagger UI
GET  /redoc    → ReDoc UI
```

RISK SCORING

AUTO Dimension Weights

Dimension	Weight
Affordability	25%

Dimension	Weight
Fees	20%
Term Length	20%
Vehicle Safety	15%
Enforcement	10%
Perfection	10%

HOUSING Dimension Weights

Dimension	Weight
Affordability	25%
Fees	20%
Lease Terms	25%
Habitability	15%
Eviction Safety	15%

Risk Levels

Score Range	Level
0–24	GREEN

Score Range	Level
25–54	YELLOW
55–100	RED

LOCAL SETUP

```
# 1. Clone and create virtual environment
python -m venv venv
source venv/bin/activate   # Windows:
venv\Scripts\activate

# 2. Install dependencies
pip install -r requirements.txt

# 3. Set environment variables
cp .env.example .env
# Edit .env: set DATABASE_URL to your local
Postgres

# 4. Create database
createdb arkheia_cps

# 5. Run migrations (or let startup auto-
create in dev mode)
alembic upgrade head

# 6. Start server
uvicorn app.main:app --reload --port 8000
```

```
# 7. Open docs
open http://localhost:8000/docs
```

.env.example

```
DATABASE_URL=postgresql://postgres:postgres
@localhost:5432/arkheia_cps
SECRET_KEY=change-me-in-production
ENV=dev
```

RENDER DEPLOYMENT

```
# 1. Push to GitHub
git init && git add . && git commit -m
"ARKHEIA-CPS initial deployment"
git remote add origin
https://github.com/yourorg/arkheia-cps.git
git push -u origin main

# 2. Connect to Render
# - New Web Service → connect GitHub repo
# - render.yaml is auto-detected
# - Render creates Postgres and wires
DATABASE_URL automatically

# 3. First deploy runs:
#    pip install -r requirements.txt
#    uvicorn app.main:app --host 0.0.0.0 --
```

```
port 8000

# 4. Run migrations after first deploy
# In Render shell:
alembic upgrade head
```

WEBFLOW INTEGRATION

Call from Webflow using `fetch` or a Webflow Logic workflow:

```
// Submit auto contract
const response = await
fetch("https://arkheia-cps-
api.onrender.com/contracts/auto", {
  method: "POST",
  headers: { "Content-Type":
"application/json" },
  body: JSON.stringify(contractPayload)
});
const analysis = await response.json();

// Display results
console.log(analysis.risk_level);
// "RED" | "YELLOW" | "GREEN"
console.log(analysis.overall_risk_score);
// 0–100
console.log(analysis.triggered_alerts);
// [{code, severity, message}]
```

```
console.log(analysis.explanations);
// [plain-language strings]
```

TESTING

Use the payloads in `sample_payloads.json`:

```
# Compliant auto contract
curl -X POST
http://localhost:8000/contracts/auto \
  -H "Content-Type: application/json" \
  -d @- < sample_payloads.json | jq
'.auto_compliant'

# Predatory contract with illegal
perfection
curl -X POST
http://localhost:8000/contracts/auto \
  -H "Content-Type: application/json" \
  -d "$(jq '.auto_predatory_perfection'
sample_payloads.json)"

# Predatory housing lease
curl -X POST
http://localhost:8000/contracts/housing \
  -H "Content-Type: application/json" \
  -d "$(jq '.housing_predatory'
sample_payloads.json)"
```

EXTENDING TO NEW STATES

1. Add state rules to `app/utils/statutes.py`:
   - `AUTO_PERFECTION_RULES["NC"] = {...}`
   - `HOUSING_TENANT_RULES["NC"] = {...}`
2. The engine picks up state rules automatically via `jurisdiction_state` on the payload.

EXTENDING TO NEW VERTICALS

1. Add FIA model in `app/models/`
2. Add Pydantic schema in `app/schemas/`
3. Add FIA service in `app/services/fia_{vertical}.py`
4. Add OIA-PEM service in `app/services/oia_pem_{vertical}.py`
5. Add RTCSA service in `app/services/rt_csa_{vertical}.py`
6. Register router in `app/main.py`

The 11-layer ARKHEIA architecture is designed for plug-in expansion.

ARKHEIA Contract Protection System · FIA → OIA-PEM → RTCSA
Built on ARKHEIA GROUP discipline architecture

Georgia O.C.G.A. statutes: §§ 10-1-390, 10-1-30, 10-1-36, 40-3-53, 44-7-1 et seq.